

Threads, exclusión mutua y locks

Técnicas de Programación Concurrente

Problema

SETI@home fue un proyecto de computación distribuida que permitía a voluntarios de todo el mundo donar la potencia de procesamiento de sus computadoras para analizar señales de radio captadas por telescopios, buscando posibles señales de inteligencia extraterrestre. Inspirados en él vamos a modelar el siguiente problema:

Un coordinador necesita procesar muestras de señales (modeladas como float). Para ello las distribuye en un conjunto de procesadores. Luego espera los resultados de cada para unirlos e iniciar la siguiente fase de procesamiento.

Modelar la situación planteada en Rust, considerando que:

- A todos los procesadores les envía el mismo valor de señal, $1/N$ del valor de la señal completa, con N la cantidad de procesadores disponibles.
- Unir los resultados implica sumarlos para generar un nuevo valor de señal.
- El procesamiento consiste en generar un número aleatorio entre 0 y 1 y multiplicarlo por el valor de señal.

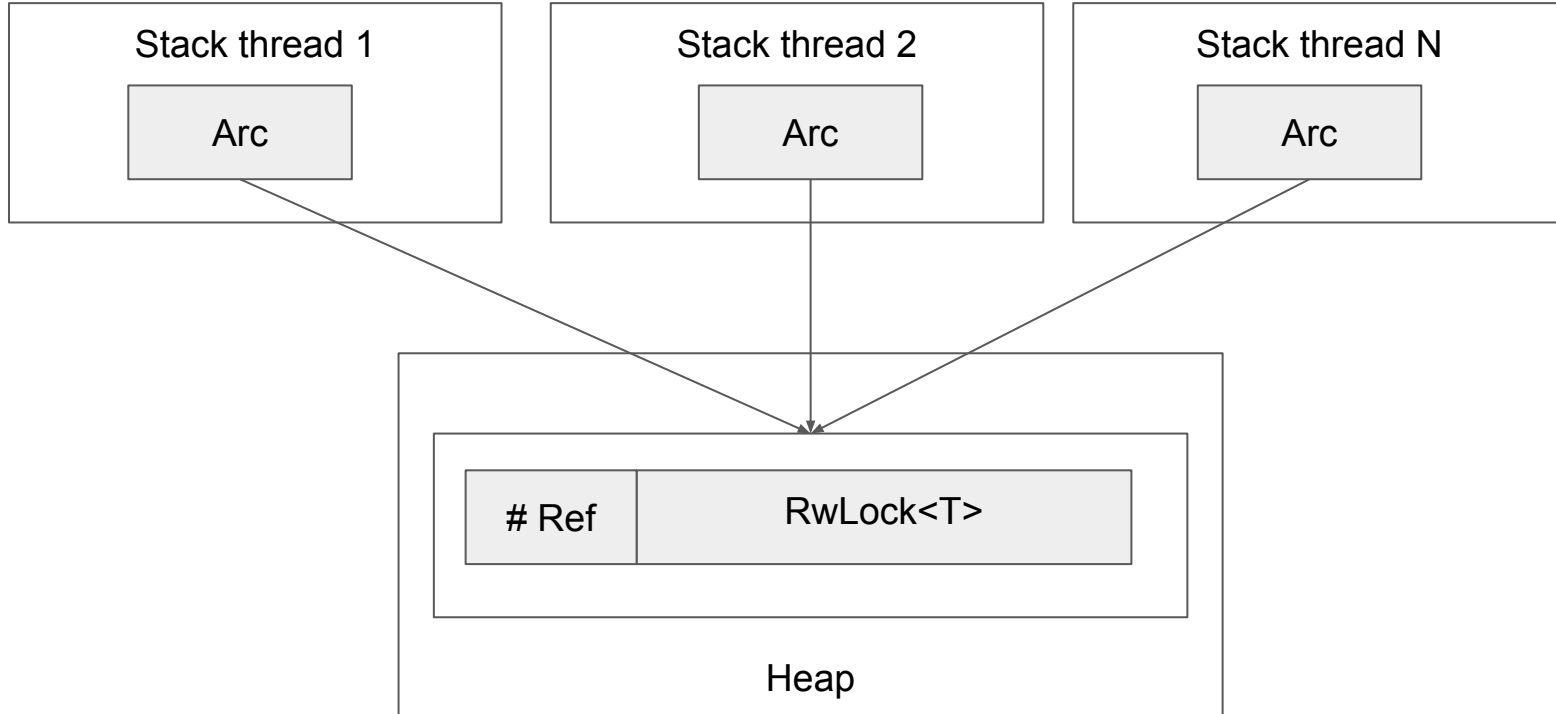
Problema 2

Hubo problemas con el proyecto SETI ya que algunos procesadores se cuelgan y nunca retornan el valor calculado de la señal, requiriendo un reinicio completo del sistema.

Para mitigarlo se decidió implementar un algoritmo alternativo:

- Los procesadores funcionarán de forma autónoma.
- Leerán el valor inicial de la señal de un repositorio conocido y tomarán $1/N$ del mismo. N es un valor conocido y constante.
- Al terminar el procesamiento, sumarán la variación de señal final al repositorio:
[resultado del procesamiento - valor inicial]

Funcionamiento de Arc



Tarea

Realizar un pequeño *idle game*

- Algunas entidades "extraerán" (generarán aleatoriamente) oro
- Algunas entidades podrán convertir oro en recursos (a gusto)
- Otras entidades podrán convertir combinaciones de recursos en + oro
- Otras entidades podrán solamente consumir recursos
- Periódicamente se reporta por pantalla el nivel de recursos y oro